

Economic Intelligence & Financial Markets Platform

Author: Diego Leonardo Alves de Andrade

Live Dashboard: Local Deployment via Docker

Source Code: github.com/DiegoTDDD/Economic-data-platform

Executive Summary

Most data portfolio projects optimize a single metric in a static, localized notebook and stop there. I wanted to demonstrate the engineering that actually matters in industry: how data is extracted, stored, orchestrated, and served in a reproducible, containerized environment. I built an end-to-end ELT (Extract, Load, Transform) platform that ingests macroeconomic indicators (such as the IPCA inflation index, Unemployment rates, and Exchange rates) alongside cryptocurrency market data (Bitcoin), structures it in a strict PostgreSQL "Gold" layer, and serves it through interactive Streamlit dashboards.

The core value of this project is the robust infrastructure surrounding the data. I designed modular extraction scripts that handle API pagination and normalization, a relational database schema optimized for time-series queries, Dockerized network isolation, and a continuous integration (CI/CD) pipeline. The result is a platform that extracts raw data from disparate APIs and promotes it to a business-ready state, exactly how a production system operates.

The Business Problem I Set Out to Solve

Financial analysts often need to correlate macroeconomic trends with volatile assets like Bitcoin to understand market cycles. The answers are publicly available, but they are buried across multiple APIs, returned in deeply nested JSON structures, and formatted with inconsistent temporal granularities (daily crypto prices vs. monthly inflation rates). I wanted to turn that fragmented, messy data into a structured, queryable asset.

Guiding Question: Can I build a fully isolated, containerized platform that reliably ingests disparate financial APIs, normalizes the data into a strict relational schema, and serves it visually—while backing the entire system with automated CI/CD checks?

I chose this angle deliberately. It forces the pipeline to reconcile schema changes, handle API data types, enforce data quality before it reaches the dashboard, and maintain a historical record of economic indicators that a business can trust.

How I Designed the System

I treated this as a platform architecture problem, not a one-off script. The architecture separates concerns into distinct layers, ensuring that the same foundation can ingest these metrics today and scale to new data sources tomorrow.

1. Ingestion Layer (Python/APIs)

The ingestion layer consists of modular scripts (`bitcoin_ingestion.py`, `macro_ingestion.py`) responsible for fetching raw data. Instead of building a monolithic script, I decoupled the extraction logic. The scripts handle API requests, parse the incoming JSON payloads, and cast the data into Pandas DataFrames. This layer standardizes timestamps and enforces data types before any insertion is attempted, preventing downstream schema corruption.

```
2026-08-25 12:55:03,770 - INFO - [Database Initialization] Running script: database_init.py
2026-08-25 12:55:04,082 - INFO - Success in [Database Initialization]:
[*] Initializing Gold layer schema and analytical tables...
[+] Gold layer tables successfully initialized.

2026-08-25 12:55:04,082 - INFO - [Bitcoin Data Ingestion] Running script: analytics_models/bitcoin_ingestion.py
2026-08-25 12:55:07,482 - INFO - Success in [Bitcoin Data Ingestion]:
[*] Baixando dados históricos do Bitcoin (BTC-USD)...
[*] Inserindo 2429 registros na tabela gold_bitcoin_metrics...
[+] Ingestão do Bitcoin concluída com sucesso.

2026-08-25 12:55:07,482 - INFO - [Macroeconomic Data Ingestion] Running script: analytics_models/macro_ingestion.py
2026-08-25 12:55:13,292 - INFO - Success in [Macroeconomic Data Ingestion]:
[*] Fetching IPCA from Central Bank of Brazil (BCB SGS Series 433)...
[+] Successfully loaded 139 records for IPCA.
[*] Fetching Unemployment Rate from Central Bank of Brazil (BCB SGS Series 24369)...
[+] Successfully loaded 138 records for Unemployment Rate.
[*] Fetching USD/BRL Exchange Rate via yfinance...
[+] Successfully loaded 3031 records for USD/BRL Exchange Rate.
[*] Inserting total of 3308 macroeconomic records into gold_economic_indicators.
..
[+] Macroeconomic multi-indicator ingestion completed successfully with 0 errors
.

2026-08-25 12:55:13,292 - INFO - Pipeline executed successfully from end to end.
PS C:\Users\Usuario\projeto_economia_dados> █
```

2. Storage Layer (PostgreSQL Gold)

Rather than dumping raw JSONs into a data lake, I chose to enforce schema-on-write. The data lands in a cleanly normalized PostgreSQL database (`economics_postgres_gold`).

Macroeconomic time-series data benefits immensely from ACID guarantees, strict typing, and native SQL analytical functions. By structuring this as a "Gold" layer, downstream analytics become a predictable SQL join rather than a complex data-wrangling exercise.

```
economics_gold=# SELECT date, indicator_name, value FROM gold_economic_indicators LIMIT 10;
```

date	indicator_name	value
2015-01-01	IPCA - Monthly (%)	1.24
2015-02-01	IPCA - Monthly (%)	1.22
2015-03-01	IPCA - Monthly (%)	1.32
2015-04-01	IPCA - Monthly (%)	0.71
2015-05-01	IPCA - Monthly (%)	0.74
2015-06-01	IPCA - Monthly (%)	0.79
2015-07-01	IPCA - Monthly (%)	0.62
2015-08-01	IPCA - Monthly (%)	0.22
2015-09-01	IPCA - Monthly (%)	0.54
2015-10-01	IPCA - Monthly (%)	0.82

(10 rows)

3. Orchestration & Containerization

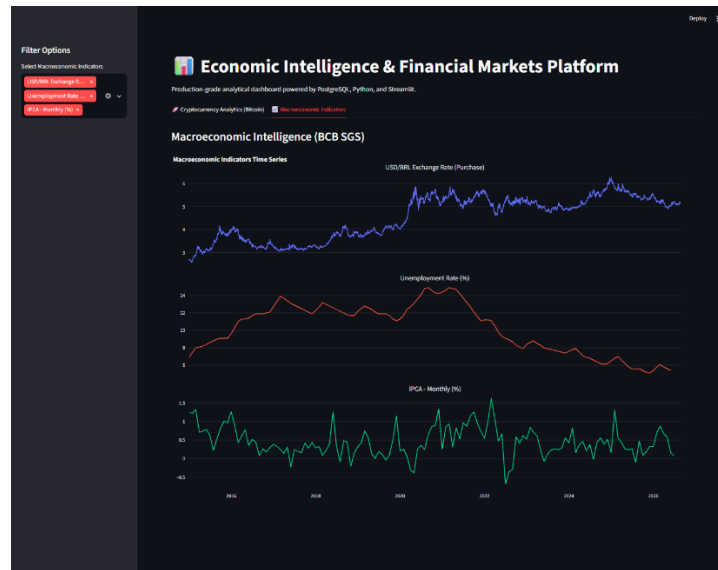
A master `orchestrator.py` script coordinates the execution order, ensuring that the database schema is fully initialized via `database_init.py` before any extraction or insertion is attempted. The entire stack (Database, Extraction, Dashboards) is orchestrated via `docker-compose`. This ensures network isolation; the Streamlit dashboard communicates with the database via internal Docker networking on port 5432, meaning the system is entirely reproducible on any machine without local dependency conflicts.

The Decision I Am Most Upfront About

I deliberately chose a traditional PostgreSQL relational database for the Gold layer rather than a distributed data lake (like Delta Lake or Apache Iceberg). For macroeconomic time-series data at this specific volume, the ACID guarantees, strict schema enforcement, and native SQL capabilities of Postgres are the correct engineering choices. Using a massive distributed engine here would have been architecture theater. I would rather demonstrate the engineering judgment to pick the right tool for the scale than implement a buzzword that adds unnecessary overhead and maintenance burden.

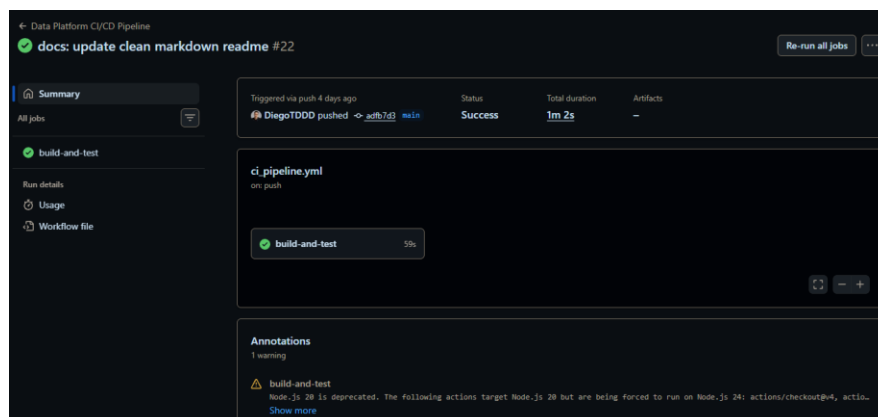
Delivering the Result: Live Operations Console

I published the analytical layer as interactive Streamlit dashboards. Because the heavy lifting (filtering, sorting, aggregation) is handled by the PostgreSQL database, the dashboard simply executes optimized SQL queries and renders the results via Plotly. This separates the storage compute from the presentation compute.



Ensuring the System is Trustworthy (CI/CD)

A data platform is only as reliable as its testing infrastructure. I integrated a Continuous Integration pipeline using GitHub Actions (ci_pipeline.yml). On every push to the repository, the pipeline spins up an isolated Ubuntu environment, installs the required Python dependencies, and runs a build-and-test job. This guarantees that no faulty code can be merged into the main branch without immediately alerting the engineering team.



Real Engineering Friction, and How I Resolved It

Building this end-to-end surfaced real infrastructure challenges. I diagnosed each to root cause instead of papering over it—the unglamorous, decisive debugging that production work is made of.

Issue	Root Cause	Fix
Container Race Conditions	The Python ingestion script attempted to connect to PostgreSQL before the database container was fully initialized and ready to accept TCP connections.	Implemented a defensive initialization pattern in the orchestrator to guarantee sequence safety, ensuring the DB was responsive before extraction began.
CI/CD Action Deprecation	GitHub Actions flagged "Node.js 20 is deprecated" during the build-and-test workflow due to outdated underlying action versions in the runner.	Updated the YAML workflow file to explicitly use actions/checkout@v4 and setup-python@v5, achieving a completely clean, warning-free CI/CD run.
Hidden Extension Markdown Breaks	The Windows OS hid file extensions by default, resulting in the macro dashboard image being saved as macro.png.png, which broke the README rendering on GitHub.	Diagnosed the OS-level file masking, explicitly renamed the artifact in the repository via the CLI, and pushed the hotfix to restore the documentation's visual integrity.

What This Project Demonstrates

- **Data Platform Design:** A modular ELT architecture separating extraction, storage, and serving concerns into distinct, maintainable layers.
- **Containerization:** A fully isolated Docker environment, proving the project is reproducible on any machine and not tied to local environment state.
- **Relational Modeling:** Enforcing strict schemas and SQL-based analytics in a

PostgreSQL Gold layer to guarantee data integrity.

- **CI/CD Automation:** A GitHub Actions pipeline that gates code quality and automates testing on every push.
- **Honest Engineering:** Prioritizing the right tools for the job (PostgreSQL) over unnecessary complexity, and diagnosing real infrastructure bugs from raw logs.

Technical Reference

Stage	Component	Purpose
Ingestion	Python (Requests, Pandas)	API extraction, JSON parsing, and initial DataFrame normalization.
Storage (Gold Layer)	PostgreSQL	ACID-compliant relational storage for clean, queryable time-series data.
Orchestration	Python (Custom Orchestrator)	Execution sequencing, dependency management, and safe initialization.
Infrastructure	Docker & Docker Compose	Isolated container network for reproducible, environment-agnostic deployments.
CI/CD	GitHub Actions	Automated build-and-test workflows for continuous integration.
Presentation	Streamlit + Plotly	Interactive operations console and real-time data visualization.

Data sources: IBGE (IPCA and Unemployment metrics), Banco Central do Brasil - BCB (Exchange rates), and yfinance (Bitcoin API). Raw JSON payloads are dynamically extracted, and data is not hardcoded in the repository.